



Sistema de Caronas UFMG



Concepção, Projeto e Desenvolvimento de uma Solução Orientada a Objetos em C++

Ricardo Rocha, Aaron Guizoli, Italo Avelar e Igor Hendrix

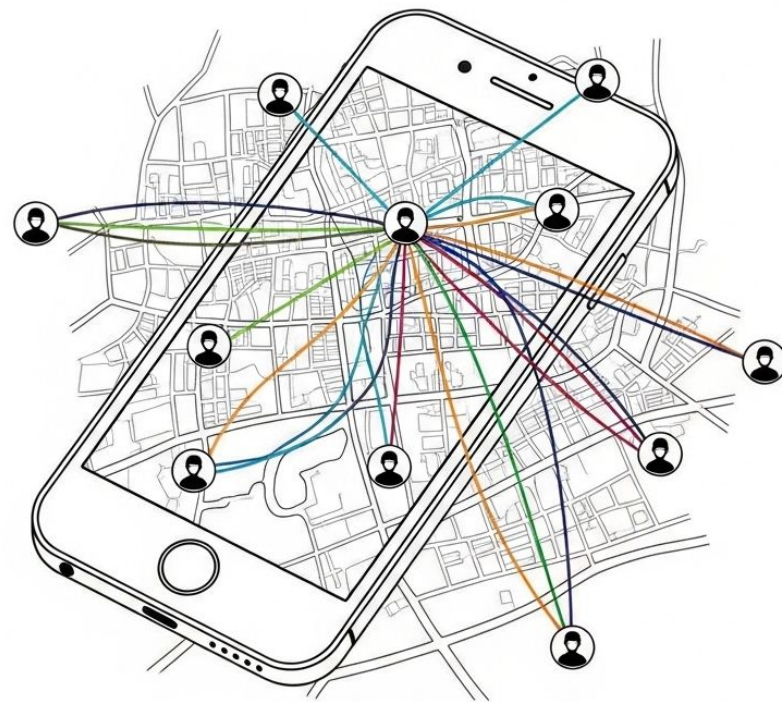
Desafios de Mobilidade na UFMG

- Ausência de plataforma segura para caronas
- Falta de organização nos grupos de caronas existentes
- Dificuldade em conectar motoristas e passageiros



Solução: App Caronas UFMG

- Desenvolvimento em C++11 com foco em Orientação a Objetos.
 - Otimiza mobilidade no campus UFMG.
 - Validação de identidade institucional via CPF.
 - Fomenta confiança com reputação.
-

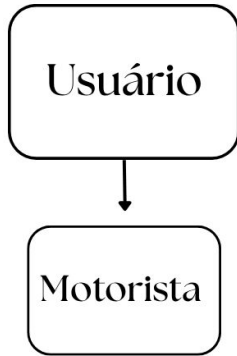


Funcionalidades Principais

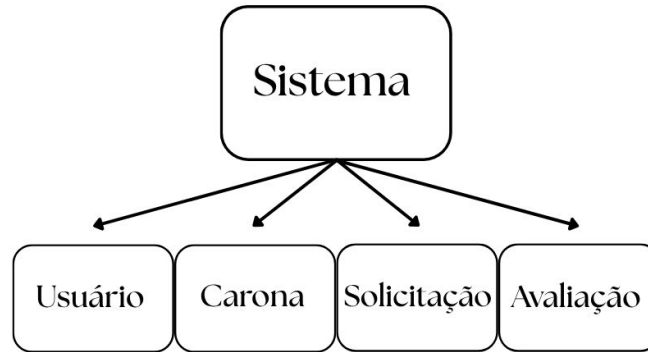
- **Como Passageiro:** Buscar caronas por zonas, solicitar participação, negociar pontos de embarque, avaliar motoristas.
- **Como Motorista:** Cadastrar veículos, oferecer caronas, gerenciar solicitações, avaliar passageiros.
- **Segurança:** Caronas exclusivas para mulheres e sistema de avaliação mútua para reputação.

Diagrama de Classes

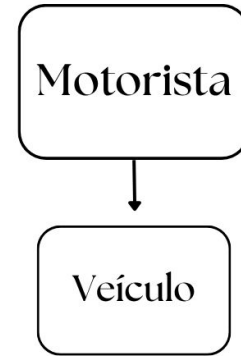
Herança



Composição



Agregação



Classe Usuário: A Base

A classe **Usuario** é a base polimórfica para todos os usuários do sistema, encapsulando dados comuns e comportamentos básicos. Ela é fundamental para a representação de perfis e autenticação.

1. Classe base para a classe derivada **Motorista**.
2. Atributos: nome, CPF, telefone, e-mail, senha, data de nascimento, gênero, tipo de vínculo UFMG.
3. Funcionalidades: verificação de senha, cálculo de média de avaliações, determinação de medalhas (reputação).
4. Gerenciar notificações e avaliações recebidas.

Classe Usuário: A Base

Encapsulamento de dados

```
1  class Usuario {
2      protected:
3          std::string _nome, _cpf, _email, _senha;
4          std::string _telefone;
5          std::string _data_nascimento;
6          Genero _genero;
7          std::vector<Avaliacao*> _avaliacoes_recebidas;
8          std::vector<Notificacao> _notificacoes;
9          std::string _vinculo_tipo;
10         std::string _detalhe_vinculo;
11
12     public:
13         Usuario(std::string nome, std::string cpf,
14                 std::string telefone, std::string data_nascimento,
15                 std::string email, std::string senha, Genero genero,
16                 std::string vinculo_tipo, std::string detalhe_vinculo);
17 }
```




```
1  std::string get_vinculo() const;
2      std::string get_vinculo_raw() const;
3      std::string get_detalhe_vinculo() const;
4
5      const std::string& get_email() const;
6      const std::string& get_senha() const;
7      Genero get_genero() const;
8
9
10     const std::string& get_cpf() const;
11     const std::string& get_nome() const;
12     bool verificar_senha(const std::string& senha) const;
13
14     virtual bool is_motorista() const;
15
16     double get_media_avaliacoes() const;
17     std::string get_medalha() const;
18     void adicionar_avaliacao_recebida(Avaliacao* avaliacao);
19     virtual void imprimir_perfil() const;
20
21     const std::string& get_telefone() const;
22     const std::string& get_data_nascimento() const;
23
24     void set_email(const std::string& email);
25     void set_telefone(const std::string& telefone);
26     void set_senha(const std::string& senha);
27     void set_genero(Genero genero);
28
29     void adicionar_notificacao(const Notificacao& notificacao);
30     const std::vector<Notificacao>& get_notificacoes() const;
31     std::vector<Notificacao>& get_notificacoes_mutavel();
32
33     };
```



```
1  bool Usuario::verificar_senha(const std::string& s) const {
2      return _senha == s;
3
4  }
5
6  double Usuario::get_media_avaliacoes() const {
7      if (_avaliacoes_recebidas.empty()) return 0.0;
8      // Soma as notas das avaliações recebidas
9      double soma = std::accumulate(_avaliacoes_recebidas.begin(), _avaliacoes_recebidas.end(), 0.0,
10          [](double acc, const Avaliacao* aval) { return acc + aval->get_nota(); });
11      return soma / _avaliacoes_recebidas.size();
12  }
13
14  std::string Usuario::get_medalha() const {
15      double media = get_media_avaliacoes();
16      if (media >= 4.5) return "Ouro";
17      if (media >= 3.0) return "Prata";
18      if (media > 0.0) return "Bronze";
19      return "Nenhuma (sem avaliacoes ou media 0)";
20  }
21
22
23  void Usuario::adicionar_notificacao(const Notificacao& notificacao) {
24      _notificacoes.push_back(notificacao); // Adiciona uma cópia da notificação
25  }
26
27  std::vector<Notificacao>& Usuario::get_notificacoes_mutavel() {
28      return _notificacoes; // Retorna uma referência mutável ao vetor
29  }
```

O Papel do Motorista: Especialização de Usuário

A classe **Motorista** herda de **Usuário**, adicionando atributos e funcionalidades específicas, como número da CNH e gerenciamento de veículos. É um exemplo chave de polimorfismo, onde um **Usuário** pode se comportar como **Motorista**

1. Herdada de **Usuário**.
2. Atributos Adicionais: `_cnh_numero`, `std::vector<Veículo*>` para veículos.
3. Funcionalidades: Adicionar/remover/buscar veículos.
4. Sobrescreve `is_motorista()` e `imprimir_perfil()` para comportamento polimórfico.



```
1  class Motorista : public Usuario {
2      private:
3          std::vector<Veiculo*> _veiculos;
4          std::string _cnh_numero;
5
6      public:
7          Motorista(std::string nome, std::string cpf, std::string telefone, std::string data_nascimento,
8                  std::string email, std::string senha, Genero genero, std::string vinculo_tipo,
9                  std::string detalhe_vinculo, std::string cnh_numero);
10
11          ~Motorista() override;
12
13          void imprimir_perfil() const override;
14
15          void adicionar_veiculo(Veiculo* veiculo);
16          bool is_motorista() const override;
17
18          const std::string& get_cnh_numero() const;
19
20          const std::vector<Veiculo*>& get_veiculos() const;
21
22          Veiculo* buscar_veiculo_por_placa(const std::string& placa) const;
23          Veiculo* buscar_veiculo_por_indice(size_t indice) const;
24
25
26          bool remover_veiculo(const std::string& placa);
27      };
28 }
29
30 #endif
```

O Papel do Motorista: Especialização de Usuário

```
1 // Usuario.hpp
2 virtual bool is_motorista() const;
3 virtual void imprimir_perfil() const;
4
5 // Usuario.cpp
6 bool Usuario::is_motorista() const { return false; }
7
8 // Motorista.hpp
9 bool is_motorista() const override;
10 void imprimir_perfil() const override;
11
12 // Motorista.cpp
13 bool Motorista::is_motorista() const { return !_cnh_numero.empty(); }
```

Gerenciamento de Caronas: Carona e CaronaFactory

A classe Carona modela as viagens oferecidas e o CaronaFactory centraliza sua criação, juntos orquestrando o fluxo das caronas no sistema.

1. Classe Carona: Representa a viagem em si.
 - a. Atributos: ID, origem, destino, data/hora, motorista, veículo, passageiros, vagas, status.
 - b. Composição: Associa Usuario, Veiculo, Solicitacao.
 - c. Métodos: set_status(), adicionar_passageiro(), exibir_info().
2. Classe CaronaFactory: Responsável pela criação de objetos Carona.
 - a. Padrão: Factory.
 - b. Método: criar_carona().
 - c. Propósito: Centraliza a criação de Caronas, simplificando o processo.
3. Interação Central: A Carona é o hub principal, conectando Solicitação e Avaliação.

Definição e implementação da classe Carona

```
1 // Em include/Carona.hpp
2 class Carona {
3 private:
4     int _id;
5     static int _proximo_id;
6     std::string _origem_nome;
7     std::string _destino_nome;
8     // ... outros atributos de zona e data/hora ...
9     Usuario* _motorista; // Composição
10    Veiculo* _veiculo_usado; // Composição
11    std::vector<Usuario*> _passageiros; // Composição
12    std::vector<Solicitacao*> _solicitacoes_pendentes; // Composição
13    int _vagas_disponiveis;
14    bool _apenas_mulheres;
15    StatusCarona _status;
16    TipoCarona _tipo;
17
18 public:
19     // ... Construtor e outros métodos públicos ...
20     int get_id() const;
21     Usuario* get_motorista() const;
22     void set_status(StatusCarona novo_status);
23     // ... outros getters e setters ...
24 };
```

```
1 // Em src/Carona.cpp
2 Carona::Carona(std::string origem_nome, std::string destino_nome, ...)
3     : _id(gerar_proximo_id()),
4       _origem_nome(origem_nome),
5       _destino_nome(destino_nome),
6       // ... inicialização de outros membros ...
7       _motorista(motorista), // Ponteiro para o Motorista
8       _veiculo_usado(veiculo_usado), // Ponteiro para o Veiculo
9       _passageiros(),
10      _solicitacoes_pendentes(),
11      _vagas_disponiveis(0),
12      _apenas_mulheres(apenas_mulheres),
13      _status(StatusCarona::AGUARDANDO),
14      _tipo(tipo) {
15     if (veiculo_usado) {
16         _vagas_disponiveis = veiculo_usado->get_lugares() - 1;
17     } // ...
18 }
```

CaronaFactory



```
1 // Em include/CaronaFactory.hpp
2 class CaronaFactory {
3 public:
4     static Carona criar_carona(std::string origem_nome, std::string destino_nome, Zona origem_zona, ...);
5 };
6
7 // Em src/CaronaFactory.cpp
8 Carona CaronaFactory::criar_carona(std::string o_nome, std::string d_nome, Zona o_zona, Zona d_zona, UFMGPosicao ufmg_pos, ...) {
9     return Carona(o_nome, d_nome, o_zona, d_zona, ufmg_pos, dt, m, vu, am, t);
10 }
11
12 // Em src/Carona.cpp
13 void Carona::adicionar_passageiro(Usuario* passageiro) {
14     if (_vagas_disponiveis > 0) {
15         _passageiros.push_back(passageiro);
16         _vagas_disponiveis--;
17     }
18 }
19
20 // Em src/Carona.cpp
21 void Carona::set_status(StatusCarona novo_status) { _status = novo_status; }
```


Negociação de Caronas: Solicitação

A classe **Solicitação** modela a requisição de um passageiro para participar de uma carona, encapsulando todo o fluxo de negociação (incluindo contrapropostas) e o rastreamento do status das avaliações pós-viagem.

1. **Status de Negociação:** PENDENTE, ACEITA, RECUSADA, AGUARDANDO_RESPOSTA_PASSAGEIRO, RECUSADA_PROPOSTA_MOTORISTA.
2. **Armazena:** Passageiro, Carona alvo, locais de embarque/desembarque (sugeridos e propostos pelo motorista).
3. **Métodos de Ação:** aceitar(), recusar(), propor_locais_motorista(), aceitar_proposta_motorista(), recusar_proposta_motorista().
4. **Flags para rastrear avaliações:** _passageiro_avalizou_motorista, _motorista_avalizou_passageiro



```
1 void Solicitacao::aceitar() {
2     _status = StatusSolicitacao::ACEITA; // Altera o status para ACEITA
3 }
4
5 void Solicitacao::recusar() {
6     _status = StatusSolicitacao::RECUSADA; // Altera o status para RECUSADA
7 }
8
9 void Solicitacao::propor_locais_motorista(std::string local_embarque_m, std::string local_desembarque_m) {
10     _local_embarque_motorista_proposto = local_embarque_m;
11     _local_desembarque_motorista_proposto = local_desembarque_m;
12     _status = StatusSolicitacao::AGUARDANDO_RESPOSTA_PASSAGEIRO; // Motorista fez proposta, aguarda passageiro
13 }
```



```
1  std::string Solicitacao::get_status_string() const {
2      switch (_status) {
3          case StatusSolicitacao::PENDENTE: return "PENDENTE";
4          case StatusSolicitacao::ACEITA: return "ACEITA";
5          case StatusSolicitacao::RECUSADA: return "RECUSADA";
6          case StatusSolicitacao::AGUARDANDO_RESPOSTA_PASSAGEIRO: return "AGUARDANDO RESPOSTA DO PASSAGEIRO";
7          case StatusSolicitacao::RECUSADA_PROPOSTA_MOTORISTA: return "RECUSADA (PROPOSTA DO MOTORISTA)";
8          default: return "DESCONHECIDO"; // Caso de segurança para estados inesperados
9      }
10 }
11
```

```
1 void Solicitacao::exibir_para_motorista() const {
2     std::cout << "Solicitacao de: " << (_passageiro ? _passageiro->get_nome() : "N/A")
3         << " | Carona ID: " << (_carona_alvo ? std::to_string(_carona_alvo->get_id()) : "N/A")
4         << " | Status: " << get_status_string() << std::endl;
5     std::cout << "  Passageiro propos: Embarque em '" << _local_embarque_passageiro
6         << "', Desembarque em '" << _local_desembarque_passageiro << "'" << std::endl;
7     if (_status == StatusSolicitacao::AGUARDANDO_RESPOSTA_PASSAGEIRO || _status == StatusSolicitacao::RECUSADA_PROPOSTA_MOTORISTA) {
8         std::cout << "  Sua proposta: Embarque em '" << _local_embarque_motorista_proposto
9             << "', Desembarque em '" << _local_desembarque_motorista_proposto << "'" << std::endl;
10    }
11 }
```

Sistema de Avaliações

A classe **Avaliação** modela o feedback pós-viagem entre usuários, encapsulando os detalhes da avaliação e sua relação com a carona e os participantes.

1. **Detalhes da Avaliação:** Armazena nota (1-5), comentário (até 100 caracteres), avaliador, avaliado, e Carona de referência.
2. **Métodos de Ação:** Construtor `Avaliação(nota, comentário, avaliador, avaliado, carona_ref)`, e métodos de acesso (getters).
3. **Rastreamento e Impacto no Perfil:** Flags na `Solicitacao` (`_passageiro_avalizou_motorista`, `_motorista_avalizou_passageiro`) para avaliação única. Contribui para `media_avaliacoes()` e medalha no perfil.

Finalizando Caronas para Avaliar



```
1 // Em src/Sistema.cpp
2 void Sistema::finalizar_carona_completa(Carona* carona_para_finalizar) {
3     if (!carona_para_finalizar) return;
4
5     carona_para_finalizar->set_status(StatusCarona::FINALIZADA);
6
7     for (Solicitacao* s : _solicitacoes) {
8         if (s->get_carona() == carona_para_finalizar && s->get_status() == StatusSolicitacao::ACEITA) {
9             enviar_notificacao(s->get_passageiro(), "A carona ID " + std::to_string(carona_para_finalizar->get_id()) +
10                 " de " + (carona_para_finalizar->get_motorista() ? carona_para_finalizar->get_motorista()->get_nome
11                 ) : "Motorista Desconhecido") + " foi FINALIZADA. Voce ja pode avalia-la!", false);
12         }
13     }
14     salvar_dados_solicitacoes();
15     salvar_dados_caronas();
16 }
17 //em carona.cpp
18 StatusCarona Carona::get_status() const { return _status; }
19 void Carona::set_status(StatusCarona novo_status) { _status = novo_status; }
20
```

Classe Avaliação: Composição e Encapsulamento

```
1 // Em include/Avaliacao.hpp
2 class Avaliacao {
3 private:
4     int _nota;
5     std::string _comentario;
6     Usuario* _avaliador;
7     Usuario* _avaliado;
8     Carona* _carona_referencia;
9 public:
10    Avaliacao(int nota, std::string comentario, Usuario* avaliador, Usuario* avaliado, Carona* carona_ref);
11 };
12
13 // Em src/Avaliacao.cpp
14 int Avaliacao::get_nota() const { return _nota; }
15 const std::string& Avaliacao::get_comentario() const { return _comentario; }
16
17 // Em include/Solicitacao.hpp
18 class Solicitacao {
19 private:
20     bool _passageiro_avaliou_motorista;
21     bool _motorista_avaliou_passageiro;
22 public:
23     void set_passageiro_avaliou_motorista(bool avaliou);
24     void set_motorista_avaliou_passageiro(bool avaliou);
25 };
```

Fluxos de Avaliação e o Polimorfismo na Exibição

```
1 // Em src/Sistema.cpp
2 void Sistema::fluxo_avaliar_carona_passageiro() {
3     std::cout << "\n--- Avaliar Caronas (Como Passageiro) ---" << std::endl;
4     std::vector<Solicitacao*> solicitacoes_para_avaliar;
5     // ... (lógica de filtragem e criação da Avaliacao)
6 }
7
8 // Em src/Sistema.cpp
9 void Sistema::fluxo_avaliar_passageiros_motorista() {
10     if (!_usuario_logado->is_motorista()) { /* ... */ return; }
11     std::cout << "\n--- Avaliar Passageiros (Como Motorista) ---" << std::endl;
12     std::vector<Solicitacao*> solicitacoes_para_avaliar;
13     // ... (lógica de filtragem e criação da Avaliacao)
14 }
15
16 // Em src/Usuario.cpp
17 void Usuario::adicionar_avaliacao_recebida(Avaliacao* a) { _avaliacoes_recebidas.push_back(a); }
18
19 // Em include/Usuario.hpp
20 class Usuario {
21     // ...
22     virtual void imprimir_perfil() const; //const override em Em include/Motorista.hpp
23     // ...
24 };
```


Conclusão: O Ciclo de Feedback e Confiança

```
1 // Em src/Usuario.cpp
2 double Usuario::get_media_avaliacoes() const {
3     if (_avaliacoes_recebidas.empty()) return 0.0;
4     double soma = std::accumulate(_avaliacoes_recebidas.begin(), _avaliacoes_recebidas.end(), 0.0,
5         [](double acc, const Avaliacao* aval) { return acc + aval->get_nota(); });
6     return soma / _avaliacoes_recebidas.size();
7 }
8
9 // Em src/Usuario.cpp
10 std::string Usuario::get_medalha() const {
11     double media = get_media_avaliacoes();
12     if (media >= 4.5) return "Ouro";
13     if (media >= 3.0) return "Prata";
14     if (media > 0.0) return "Bronze";
15     return "Nenhuma (sem avaliacoes ou media 0)";
16 }
17
18 // Em src/Usuario.cpp (dentro de Usuario::imprimir_perfil)
19 void Usuario::imprimir_perfil() const {
20     // ... (outras informações do perfil)
21     std::cout << "Avaliacao Media: " << std::fixed << std::setprecision(1) << get_media_avaliacoes() << " estrelas" << std::endl;
22     std::cout << "Medalha: " << get_medalha() << std::endl;
23     // ...
24 }
```

Sistema

A implementação Sistema é o orquestrador central da aplicação, gerenciando o estado, os dados (usuários, caronas, solicitações, avaliações) e os fluxos de interação do usuário. Responsável pela inicialização, persistência de dados e controle do loop principal.

- 1. Possui objetos alocados dinamicamente (Usuario*, Solicitacao*, Avaliacao*).**
- 2. Gerencia carregamento e salvamento de dados em arquivos (usuarios.txt, veiculos.txt, caronas.txt, solicitacoes.txt, avaliacoes.txt).**
- 3. Contém a lógica dos fluxos de usuário (cadastro, login, oferta/solicitação de carona, etc.).**
- 4. Possui métodos utilitários para entrada/saída e conversão de dados.**

Fluxos e Persistência de dados

```
1  std::cout << "\n(1) Cadastrar Novo Veiculo | (2) Editar Veiculo | (3) Excluir Veiculo | (0) Voltar" << std::endl;
2      comando = coletar_int_input("> ", 0, 3);
3
4      try {
5          if (comando == 1) {
6              fluxo_cadastrar_veiculo();
7          } else if (comando == 2) {
8              fluxo_editar_veiculo(motorista_logado);
9          } else if (comando == 3) {
10                 fluxo_excluir_veiculo(motorista_logado);
11            }
12        }
```

```

1 void Sistema::fluxo_cadastrar_veiculo() {
2     if (!usuario_logado->is_motorista()) {
3         std::cout << "Voce nao esta registrado como motorista. Nao e possivel cadastrar veiculo." << std::endl;
4         return;
5     }
6
7     Motorista* motorista_logado = dynamic_cast<Motorista*>(_usuario_logado);
8     if (!motorista_logado) {
9         std::cerr << "ERRO INTERNO: Usuario logado deveria ser motorista mas nao pode ser convertido." << std::endl;
10        return;
11    }
12
13    std::string placa, marca, modelo, cor;
14    int lugares;
15
16    char cadastrar_outro = 's';
17    while (cadastrar_outro == 's' || cadastrar_outro == 'S') {
18        std::cout << "\n--- Cadastrar Novo Veiculo ---" << std::endl;
19        std::cout << "Placa: "; std::getline(std::cin, placa);
20        if (motorista_logado->buscar_veiculo_por_placa(placa) != nullptr) {
21            std::cout << "Voce ja possui um veiculo com esta placa." << std::endl;
22            std::cout << "Deseja tentar com outra placa? (s/n): ";
23            char tentar_outra;
24            std::cin >> tentar_outra;
25            std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
26            if (!(tentar_outra == 's' || tentar_outra == 'S')) {
27                cadastrar_outro = 'n';
28            }
29            continue;
30        }
31
32        std::cout << "Marca: "; std::getline(std::cin, marca);
33        std::cout << "Modelo: "; std::getline(std::cin, modelo);
34        std::cout << "Cor: "; std::getline(std::cin, cor);
35        std::cout << "Total de lugares (com motorista): ";
36        lugares = coletar_int_input("", 2, 99);
37
38        motorista_logado->adicionar_veiculo(new Veiculo(placa, marca, modelo, cor, lugares));
39        salvar_dados_veiculos();
40        std::cout << "Veiculo cadastrado com sucesso!" << std::endl;
41
42        std::cout << "Deseja cadastrar outro veiculo? (s/n): ";
43        std::cin >> cadastrar_outro;
44        std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
45    }
46 }

```

```

1 void Sistema::salvar_dados_veiculos() {
2     std::ofstream arquivo_veiculos("veiculos.txt", std::ios::trunc);
3     if (!arquivo_veiculos.is_open()) {
4         std::cerr << "ERRO: Nao foi possivel abrir o arquivo veiculos.txt para salvar dados." << std::endl;
5         return;
6     }
7
8     for (const auto& u : _usuarios) {
9         if (u->is_motorista()) {
10             Motorista* m_ptr = dynamic_cast<Motorista*>(u);
11             if (m_ptr) {
12                 for (const auto& veic : m_ptr->get_veiculos()) {
13                     if (veic) {
14                         arquivo_veiculos << u->get_cpf() << ";"
15                             << veic->get_placa() << ";"
16                             << veic->get_marca() << ";"
17                             << veic->get_modelo() << ";"
18                             << veic->get_cor() << ";"
19                             << veic->get_lugares()
20                             << std::endl;
21                     }
22                 }
23             }
24         }
25     }
26     arquivo_veiculos.close();
27 }

```

```

1 11122233344;HGH-8899;Volkswagen;Gol;Prata;5
2 22233344455;ABC-1234;Ford;Ranger;Branco;4
3 22233344455;XYZ-5678;Honda;Civic;Preto;5
4 33344455566;DEF-9012;Chevrolet;Onix;Azul;5
5 55566677788;GHI-3456;Toyota;Corolla;Cinza;5
6 77788899900;JKL-7890;Hyundai;HB20;Vermelho;5
7 99900011122;MNO-1234;Fiat;Mobi;Branco;4
8

```

Tratamento de exceções

```
1  if (!u) {  
2      auto dados_ufmg = buscar_dados_ufmg_por_cpf(cpf);  
3      if (std::get<0>(dados_ufmg)) {  
4          throw AppExcecao("Voce ainda nao se cadastrou no sistema de caronas. Por favor, faca seu cadastro.");  
5      } else {  
6          throw AutenticacaoFalhouException();  
7      }  
8  }
```

Garantia de Qualidade: Testes Automatizados

Durante o desenvolvimento, utilizamos o framework doctest para implementar testes de unidade. Há pelo menos uma classe de testes para cada uma das principais classes do sistema, garantindo a corretude e a robustez do código.

1. Framework: doctest para testes de unidade.
2. Testes para classes de entidade (ex: Usuario, Carona, Solicitacao).
3. Garantiu a funcionalidade correta e a ausência de bugs.

```

1  TEST_CASE("Testes da Classe Usuario") {
2      // Um sub-caso para testar o construtor e getters básicos
3      SUBCASE("Construtor e Getters Básicos") {
4          Usuario user("Joao Silva", "12345678900", "31999998888", "01/01/2000", "joao@email.com", "senha123", Genero::MASCULINO, "aluno", "Ciencia da Computacao");
5
6          // Asserções para verificar se os dados foram armazenados corretamente
7          CHECK(user.get_nome() == "Joao Silva");
8          CHECK(user.get_cpf() == "12345678900");
9          CHECK(user.get_telefone() == "31999998888");
10         CHECK(user.get_email() == "joao@email.com");
11         CHECK(user.verificar_senha("senha123") == true); // Verifica a senha
12         CHECK(user.get_genero() == Genero::MASCULINO);
13     }
14
15     // Outro sub-caso para testar a funcionalidade de verificar senha
16     SUBCASE("Verificar Senha") {
17         Usuario user("Maria", "11122233344", "987654321", "05/05/1995", "maria@email.com", "maria123", Genero::FEMININO, "funcionario", "DCC");
18
19         CHECK(user.verificar_senha("maria123") == true); // Senha correta
20         CHECK(user.verificar_senha("senhaerrada") == false); // Senha incorreta
21     }
22 }
23
24 // Define um caso de teste para a classe Carona
25 TEST_CASE("Testes da Classe Carona") {
26     // Prepara objetos de dependência para o teste da Carona
27     Motorista motorista_teste("Motorista Teste", "12312312300", "319111112222", "10/01/1985", "mot@email.com", "pass", Genero::MASCULINO, "funcionario", "DCC", "123456789");
28     Veiculo veiculo_teste("ABC1D23", "Toyota", "Corolla", "Prata", 4);
29
30     // Sub-caso para testar o construtor e getters da Carona
31     SUBCASE("Construtor e Getters") {
32         Carona carona1("Centro", "Pampulha", Zona::CENTRO_SUL, Zona::PAMPULHA, UFMGPosicao::DESTINO, "20/07/2025 14:00", &motorista_teste, &veiculo_teste, false, TipoCarona::AGENDADA);
33
34         CHECK(carona1.get_origem() == "Centro");
35         CHECK(carona1.get_destino() == "Pampulha");
36         CHECK(carona1.get_motorista()->get_nome() == "Motorista Teste");
37         CHECK(carona1.get_veiculo_usado()->get_placa() == "ABC1D23");
38         CHECK(carona1.get_vagas_disponiveis() == 3); // Veiculo com 4 lugares - 1 para motorista = 3 vagas
39     }
40 }

```